SFWRENG 3S03

Assignment 2

Matthew Farah, ⟨farahm11@mcmaster.ca⟩, 400468251

March 27, 2026

Contents

Question 1 - Basic deployment pipeline – Modeling with Petri Nets **2**

Question 2 - Safety and other useful properties – Formalizing requirements **4**
Part 2.a 4
Part 2.b 6
Part 2.c 9
Part 2.d 11

Question 3 - Critical updates – Working with priorities **16**

Question 4 - Formalizing Requirements **18**
Part 4.a 18
Part 4.b 20
Part 4.c 24

Question 5 - Reflection **28**

[3.5 marks]
Question 1

Incoming update requests are submitted to the deployment service. Each request corresponds to a deployment batch, which must pass through a sequence of processing stages before being installed on devices.

The pipeline includes the following shared resources:

- Validation Service – verifies update packages before deployment.
- Deployment Manager – coordinates sending updates to devices.
- Two Deployment Workers – workers that push updates to devices in parallel.
- Staging buffer – can hold at most three deployment batches simultaneously.

The validation service, the deployment manager, and the deployment workers process only one batch at a time.

Each deployment batch goes through the following stages:

- Submission – an update request enters the system.
- Staging – the batch waits in the staging buffer.
- Validation – the validation service checks the update package.
- Dispatch – the deployment manager assigns the batch to an available worker.
- Deployment – a worker performs the update rollout.
- Completion – the batch finishes deployment successfully.

Model the system in a Petri net.

Solution:

My Petri net model for the basic deployment pipeline is shown in the figure below. I will briefly describe the key design decisions.

- The Update_Requests place has an initial marking of 1 with a self-loop on the Submit_Request transition, allowing an unbounded number of update requests to enter the system over time.
- The staging buffer capacity of 3 is enforced by an Available_Space_In_Buffer resource place initialized to 3. The Stage_Deployment_Batch transition consumes one token from this place, and the Validate_Deployment_Batch transition returns it, ensuring at most 3 batches occupy the buffer at any time.

- The Validation_Service place (initialized to 1) enforces mutual exclusion on validation. It is consumed by Validate_Deployment_Batch and returned by either Deployment_Batch_Is_Valid or Deployment_Batch_Is_Invalid, ensuring only one batch is validated at a time.
- The Deployment_Manager place (initialized to 1) enforces mutual exclusion on dispatch. It is consumed and returned by the Assign_To_Worker transitions.
- Worker_1 and Worker_2 (each initialized to 1) model the two deployment workers. When a batch is assigned to a worker, the worker token moves to a Worker_Busy place and is returned upon rollout completion.
- After validation, a nondeterministic choice between Deployment_Batch_Is_Valid and Deployment_Batch_Is_Invalid models the possibility of invalid batches.

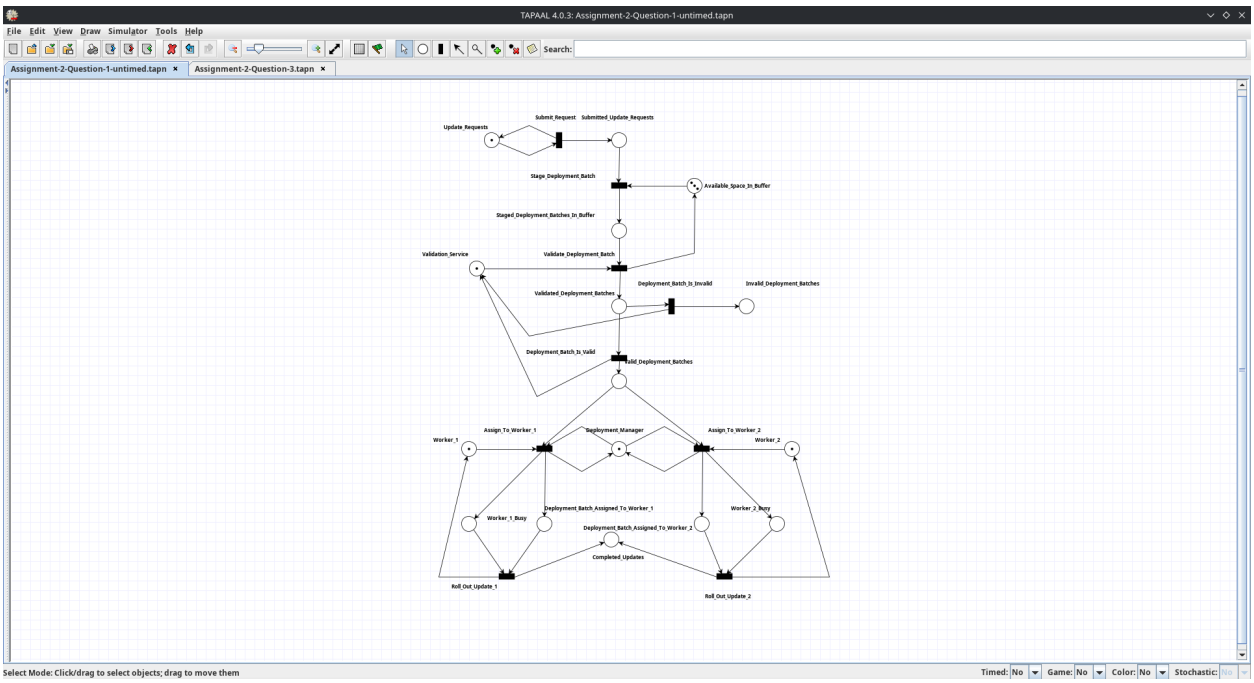

Figure 1: Petri net model for the basic deployment pipeline.

[3.5 marks]
Question 2

Formulate CTL queries to capture the following requirements. For each requirement:

1. Formulate the CTL expression and explain it in your report;
2. Implement it in TAPAAL;
3. Disclose the result of the query in your report and interpret it.

(a) Buffer safety – The system never exceeds the staging buffer capacity.

Solution:

(I) CTL Expression

The staging buffer has a capacity of 3. Thus, we need to verify that in every reachable state along every possible execution, the number of tokens in the Staged_Deployment_Batches_In_Buffer place never exceeds 3. This is a universal safety property, naturally expressed using AG:

$$AG\,(\text{TAPN1.Staged_Deployment_Batches_In_Buffer} \leq 3)$$

This reads: "for all paths and all states along those paths, the number of batches in the staging buffer is at most 3."

(II) TAPAAL Result

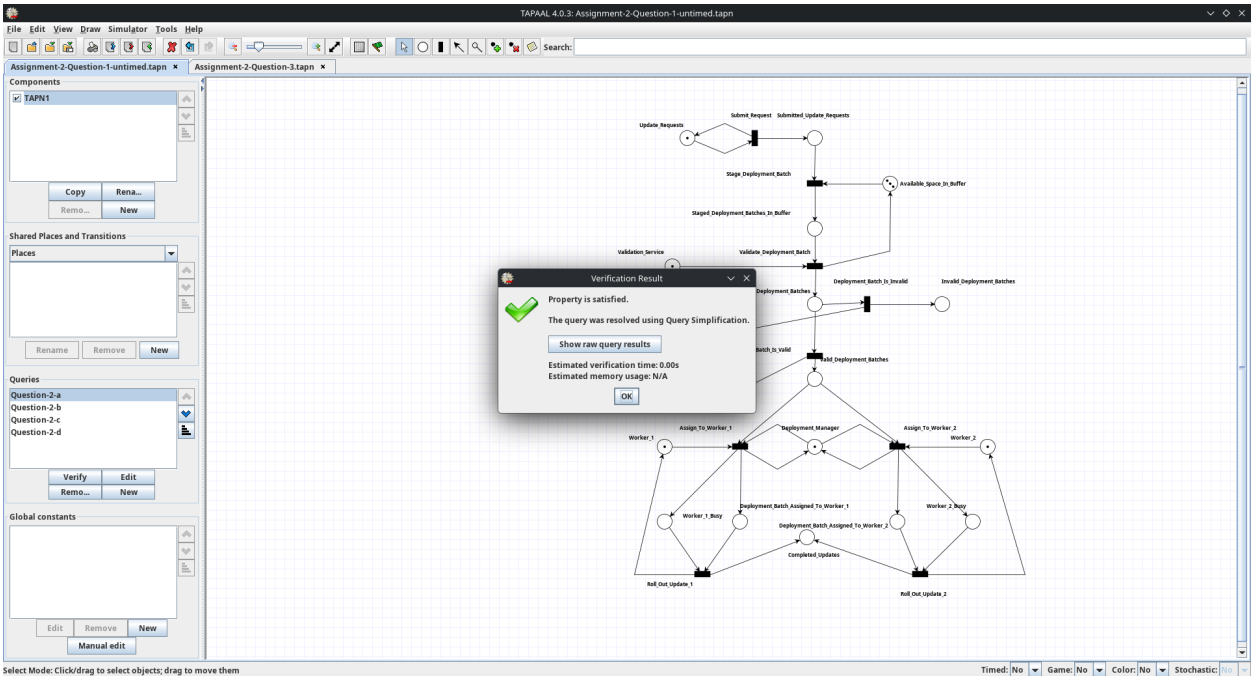

Figure 2: TAPAAL verification result for buffer safety (Q2a).

(III) Raw Query Output

```
Parameters: --k-bound 12 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta7748863151168961347.xml /tmp/verifyta12514745640973838986.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=12,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: AG (TAPN1_Staged_Deployment_Batches_In_Buffer <= 3)
RWSTATS PRE:0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
RWSTATS POST:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

Query after reduction: true
Query reduction finished after 0.000209 seconds.
Query size reduced from 4 to 0 nodes ( 100 percent reduction).
FORMULA Question-2-a TRUE TECHNIQUES COLLATERAL_PROCESSING STRUCTURAL_REDUCTION
    QUERY_REDUCTION SAT_SMT LP_APPROX
Query index 0 was solved
```

```
Query is satisfied.

Query solved by Query Simplification.

Size of net before structural reductions: 16 places, 9 transitions
Size of net after structural reductions: 0 places, 0 transitions
Structural reduction finished after 1.9e-05 seconds

Net reduction is enabled.
Removed transitions: 9
Removed places: 16
Applications of rule A: 0
Applications of rule B: 0
Applications of rule C: 0
Applications of rule D: 0
Applications of rule E: 0
Applications of rule F: 0
Applications of rule G: 0
Applications of rule H: 0
Applications of rule I: 1
Applications of rule J: 0
Applications of rule K: 0
Applications of rule L: 0
Applications of rule M: 0
Applications of rule N: 0
Applications of rule O: 0
Applications of rule P: 0
Applications of rule Q: 0
Applications of rule R: 0
Applications of rule S: 0
```

(IV) Interpretation

The property is satisfied. This confirms that the Available_Space_In_Buffer resource place (initialized to 3) correctly enforces the buffer capacity. Since the Stage_Deployment_Batch transition requires a token from this resource place to fire, and the token is only returned after validation completes, it is structurally impossible for more than 3 batches to occupy the buffer simultaneously.

(b) Resource exclusivity – The validation service and deployment manager are never used by more than one batch at the same time.

Solution:

(I) CTL Expression

To verify this, I check that the resource tokens for both the validation service and the deployment manager are conserved in a way that prevents concurrent use. Specifically, I verify two things simultaneously:

- The Deployment_Manager place always contains exactly 1 token (it is never consumed without being returned).

- The sum of tokens in Validation_Service and Validated_Deployment_Batches is always exactly 1. This captures the invariant that at any given time, the validation service is either idle (token in Validation_Service) or processing exactly one batch (token in Validated_Deployment_Batches), but never both and never neither.

The CTL expression is:

$$AG\Big(\text{TAPN1.Deployment_Manager} = 1 \\ \wedge\ \text{TAPN1.Validation_Service} + \text{TAPN1.Validated_Deployment_Batches} = 1\Big)$$

(II) TAPAAL Result

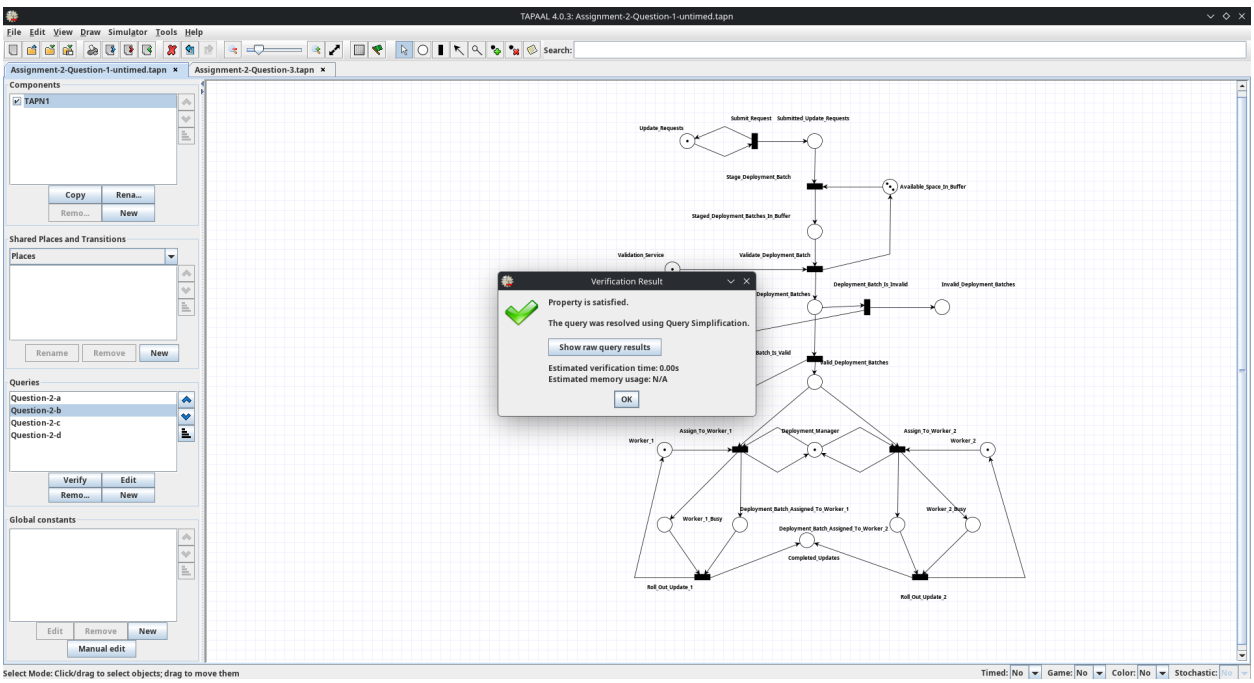

Figure 3: TAPAAL verification result for resource exclusivity (Q2b).

(III) Raw Query Output

```
Parameters: --k-bound 12 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta6815580675050042353.xml /tmp/verifyta16647063816389027138.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=12,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
```

```
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: AG ((TAPN1_Deployment_Manager == 1) and ((0 + TAPN1_Validation_Service
     + TAPN1_Validated_Deployment_Batches) == 1))
RWSTATS PRE:0,1,0,0,0
```

component is the validation service invariant, which genuinely captures mutual exclusion: the token is consumed by Validate_Deployment_Batch and only returned later by either Deployment_Batch_Is_Valid or Deployment_Batch_Is_Invalid, ensuring only one batch undergoes validation at a time.

(c) Deadlock possibility – Determine whether the system can reach a marking where no transitions are enabled.

Solution:

(I) CTL Expression

The question asks us to determine whether the system can reach a deadlock. A deadlocked marking is one where no transitions are enabled. To verify deadlock freedom, I check that in every reachable state, the system is never deadlocked:

$$AG\,(\neg \text{deadlock})$$

This reads: "for all paths and all states along those paths, the system is never in a deadlocked state." If the property is satisfied, the system is deadlock-free.

(II) TAPAAL Result

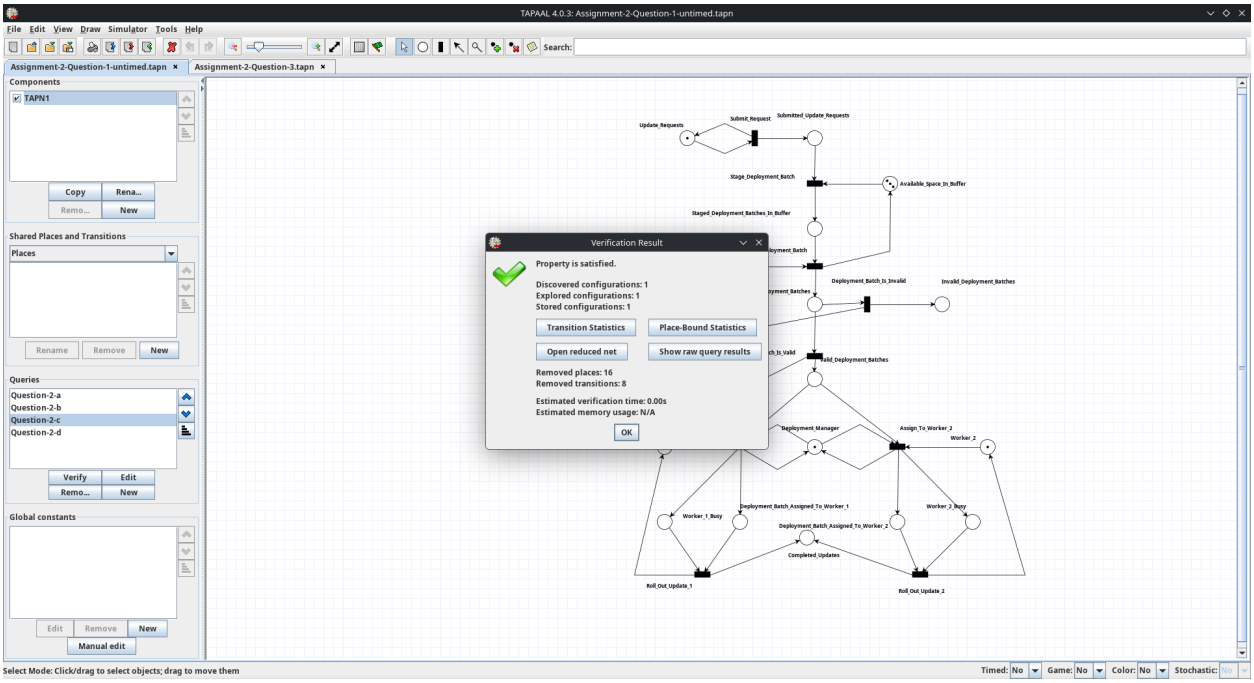

Figure 4: TAPAAL verification result for deadlock possibility (Q2c).

(III) Raw Query Output

```
Parameters: --k-bound 12 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta12240909929844012432.xml /tmp/verifyta3849426049881814372.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=12,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: AG (not deadlock)
RWSTATS PRE:0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,
RWSTATS POST:0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,

Query after reduction: (not EF deadlock)
Query reduction finished after 7e-06 seconds.
Query size reduced from 3 to 3 nodes ( 0 percent reduction).
Size of net before structural reductions: 16 places, 9 transitions
Size of net after structural reductions: 0 places, 1 transitions
Structural reduction finished after 5.5e-05 seconds
```

```
Net reduction is enabled.
Removed transitions: 8
Removed places: 16
Applications of rule A: 0
Applications of rule B: 6
Applications of rule C: 2
Applications of rule D: 2
Applications of rule E: 0
Applications of rule F: 2
Applications of rule G: 0
Applications of rule H: 0
Applications of rule I: 0
Applications of rule J: 0
Applications of rule K: 0
Applications of rule L: 0
Applications of rule M: 6
Applications of rule N: 0
Applications of rule O: 0
Applications of rule P: 0
Applications of rule Q: 0
Applications of rule R: 0
Applications of rule S: 0
FORMULA Question-2-c TRUE TECHNIQUES COLLATERAL_PROCESSING STRUCTURAL_REDUCTION
    QUERY_REDUCTION SAT_SMT EXPLICIT STATE_COMPRESSION STUBBORN_SETS
Query index 0 was solved

Query is satisfied.

STATS:
    discovered states: 1
    explored states:   1
    expanded states:   1
    max tokens:        0

TRANSITION STATISTICS
<TAPN1_Submit_Request:0><TAPN1_Assign_To_Worker_2:?><TAPN1_Stage_Deployment_Batch:?><
    TAPN1_Validate_Deployment_Batch:?><TAPN1_Assign_To_Worker_1:?><TAPN1_Roll_Out_Update_1:?><
    TAPN1_Roll_Out_Update_2:?><TAPN1_Deployment_Batch_Is_Valid:?><
    TAPN1_Deployment_Batch_Is_Invalid:?>

PLACE-BOUND STATISTICS
<TAPN1_Deployment_Batch_Assigned_To_Worker_2;?><TAPN1_Worker_1_Busy;?><TAPN1_Worker_2;?><
    TAPN1_Available_Space_In_Buffer;?><TAPN1_Validated_Deployment_Batches;?><
    TAPN1_Valid_Deployment_Batches;?><TAPN1_Worker_1;?><TAPN1_Deployment_Manager;?><
    TAPN1_Validation_Service;?><TAPN1_Deployment_Batch_Assigned_To_Worker_1;?><
    TAPN1_Staged_Deployment_Batches_In_Buffer;?><TAPN1_Submitted_Update_Requests;?><
    TAPN1_Worker_2_Busy;?><TAPN1_Invalid_Deployment_Batches;?><TAPN1_Update_Requests;?><
    TAPN1_Completed_Updates;?>

Spent 0.00055 on verification
```

(IV) Interpretation

The property is satisfied, meaning the system is deadlock-free. This is to be expected: the Submit_Request transition has a self-loop on the Update_Requests place (which is initialized to 1), so this transition is always enabled regardless of the state of the rest of the system. Therefore, the system can never reach a state in which no transitions are enabled.

(d) Pipeline progress – If a batch enters the system, is it always possible for it to eventually complete deployment?

Solution:

(I) CTL Expression

This is a liveness property asking whether, from any state in which a batch has been submitted, it is possible (not necessarily guaranteed) for the batch to eventually reach completion. The ideal CTL formulation is:

$$AG\,(\text{TAPN1.Submitted_Update_Requests} = 0 \;\vee\; EF\,(\text{TAPN1.Completed_Updates} \geq 1))$$

This encodes the implication: "in every reachable state, if a batch has been submitted, then there exists a path from that state along which some batch eventually completes." The implication $P \rightarrow Q$ is encoded as $\neg P \vee Q$, where P is "a batch has been submitted" and Q is "completion is eventually reachable."

However, this formula contains nested path quantifiers (AG wrapping EF), which can be computationally expensive for the model checker. As a practical alternative, we can verify the non-nested formula:

$$EF\,(\text{TAPN1.Completed_Updates} \geq 1)$$

This simply asks: "is it possible for a batch to ever reach the Completed_Updates place?"

(II) TAPAAL Result

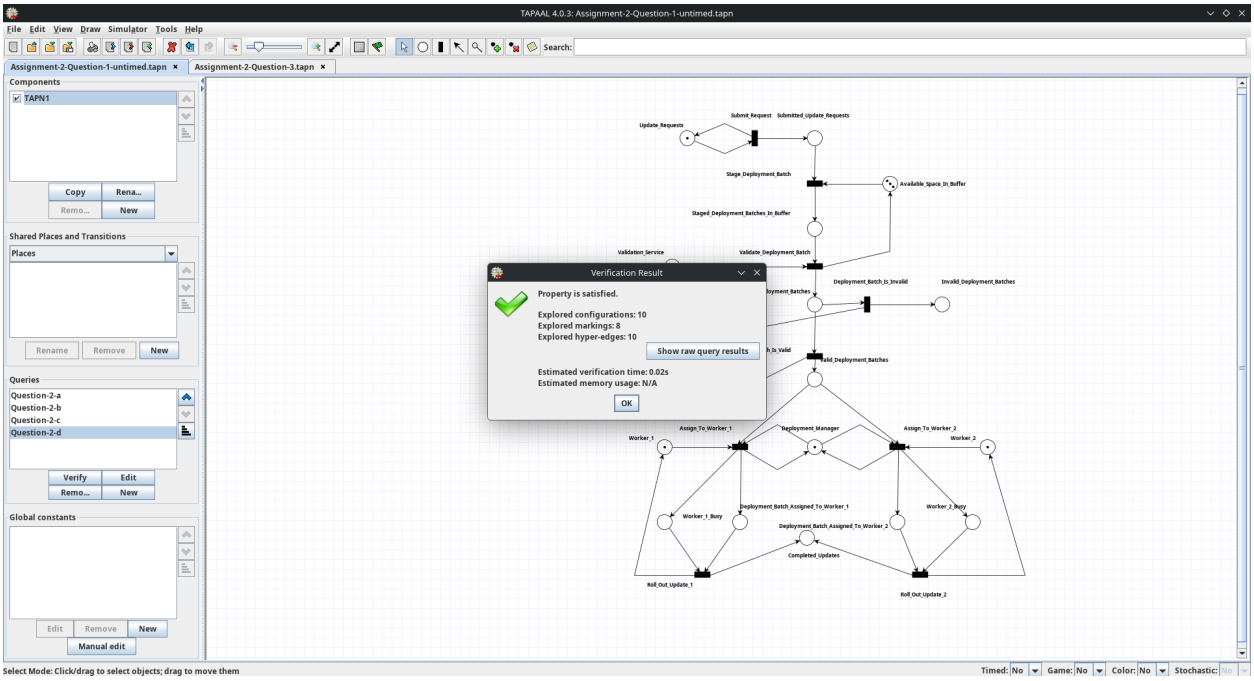

Figure 5: TAPAAL verification result for pipeline progress (Q2d).

(III) Raw Query Output

```
Parameters: --k-bound 12 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta90179630045045479.xml /tmp/verifyta13166914849516301058.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=12,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: EF (1 <= TAPN1_Completed_Updates)
RWSTATS PRE:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
RWSTATS POST:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

Query after reduction: EF (1 <= TAPN1_Completed_Updates)
Query reduction finished after 0.000187 seconds.
Query size reduced from 4 to 4 nodes ( 0 percent reduction).
Size of net before structural reductions: 16 places, 9 transitions
Size of net after structural reductions: 1 places, 2 transitions
Structural reduction finished after 4.9e-05 seconds
```

```
Net reduction is enabled.
Removed transitions: 7
Removed places: 15
Applications of rule A: 0
Applications of rule B: 3
Applications of rule C: 2
Applications of rule D: 1
Applications of rule E: 0
Applications of rule F: 0
Applications of rule G: 0
Applications of rule H: 0
Applications of rule I: 1
Applications of rule J: 0
Applications of rule K: 0
Applications of rule L: 0
Applications of rule M: 6
Applications of rule N: 0
Applications of rule O: 0
Applications of rule P: 0
Applications of rule Q: 0
Applications of rule R: 0
Applications of rule S: 4
FORMULA Question-2-d TRUE TECHNIQUES COLLATERAL_PROCESSING STRUCTURAL_REDUCTION
    QUERY_REDUCTION SAT_SMT EXPLICIT STATE_COMPRESSION STUBBORN_SETS
Query index 0 was solved

Query is satisfied.

STATS:
    discovered states: 2
    explored states:   2
    expanded states:   0
    max tokens:        1

TRANSITION STATISTICS
<TAPN1_Assign_To_Worker_2:1><TAPN1_Assign_To_Worker_1:0><TAPN1_Submit_Request:?><
    TAPN1_Stage_Deployment_Batch:?><TAPN1_Validate_Deployment_Batch:?><TAPN1_Roll_Out_Update_1
    :?><TAPN1_Roll_Out_Update_2:?><TAPN1_Deployment_Batch_Is_Valid:?><
    TAPN1_Deployment_Batch_Is_Invalid:?>

PLACE-BOUND STATISTICS
<TAPN1_Completed_Updates;1><TAPN1_Deployment_Batch_Assigned_To_Worker_2;?><TAPN1_Worker_1_Busy
    ;?><TAPN1_Worker_2;?><TAPN1_Available_Space_In_Buffer;?><
    TAPN1_Validated_Deployment_Batches;?><TAPN1_Valid_Deployment_Batches;?><TAPN1_Worker_1;?><
    TAPN1_Deployment_Manager;?><TAPN1_Validation_Service;?><
    TAPN1_Deployment_Batch_Assigned_To_Worker_1;?><TAPN1_Staged_Deployment_Batches_In_Buffer
    ;?><TAPN1_Submitted_Update_Requests;?><TAPN1_Worker_2_Busy;?><
    TAPN1_Invalid_Deployment_Batches;?><TAPN1_Update_Requests;?>

Spent 0.000529 on verification
```

(IV) Interpretation

The property is satisfied, confirming that it is indeed possible for a batch to complete deployment. Combined with the deadlock freedom result from part (c), which guarantees the system never gets stuck, we can reason that progress is always possible. Since the system never deadlocks and the completion state is reachable, there is no state from which a batch is permanently blocked from completing.

Note, however, that this does not guarantee that any specific batch will complete. Due to the nondeterministic nature of the system (new requests can continually arrive, and the

valid/invalid choice is nondeterministic), there exist execution paths in which a particular batch is indefinitely delayed. What we can say is that completion is always possible, not that it is guaranteed, a subtle but important distinction between EF and AF.

[3 marks]
Question 3

In practice, some updates must be deployed with higher urgency than regular feature updates. Pertinent examples include security and safety patches.

Extend your model to be able to model two types of deployment batches: regular and critical updates. The system must enforce the following operational policy: whenever at least one critical update batch is waiting to be dispatched, the deployment manager must assign it to a worker before any regular update batch.

Implement this policy in your Petri net using inhibitor arcs. (The current version of TAPAAL does not support priority transitions.)

Disclose your extended model in your report and identify the key changes.

Solution:

My extended Petri net model is shown in the figure below. The key changes from the basic model in Question 1 are as follows:

- The pipeline has been split into two parallel paths: one for regular update requests and one for critical update requests. Each type has its own submission source, submitted requests place, staging transition, buffer place, validation transition, validated batches place, and valid/invalid decision.
- The staging buffer, validation service, deployment manager, and workers remain shared between the two paths. In particular, the Available_Space_In_Buffer place (initialized to 3) is consumed by both regular and critical staging transitions, and returned by both validation transitions.
- The dispatch phase now has four assignment transitions, two for regular batches (one per worker) and two for critical batches (one per worker).
- Two inhibitor arcs have been added from the Valid_Critical_Deployment_Batches place to both regular assignment transitions. These arcs ensure that whenever at least one valid critical batch is waiting for dispatch, the transitions that assign regular batches to workers are disabled. This enforces the priority policy without requiring TAPAAL's unsupported priority transitions.
- The downstream portion of the pipeline (rollout and completion) remains shared, since once a batch has been assigned to a worker, the distinction between regular and critical no longer matters for deployment purposes.

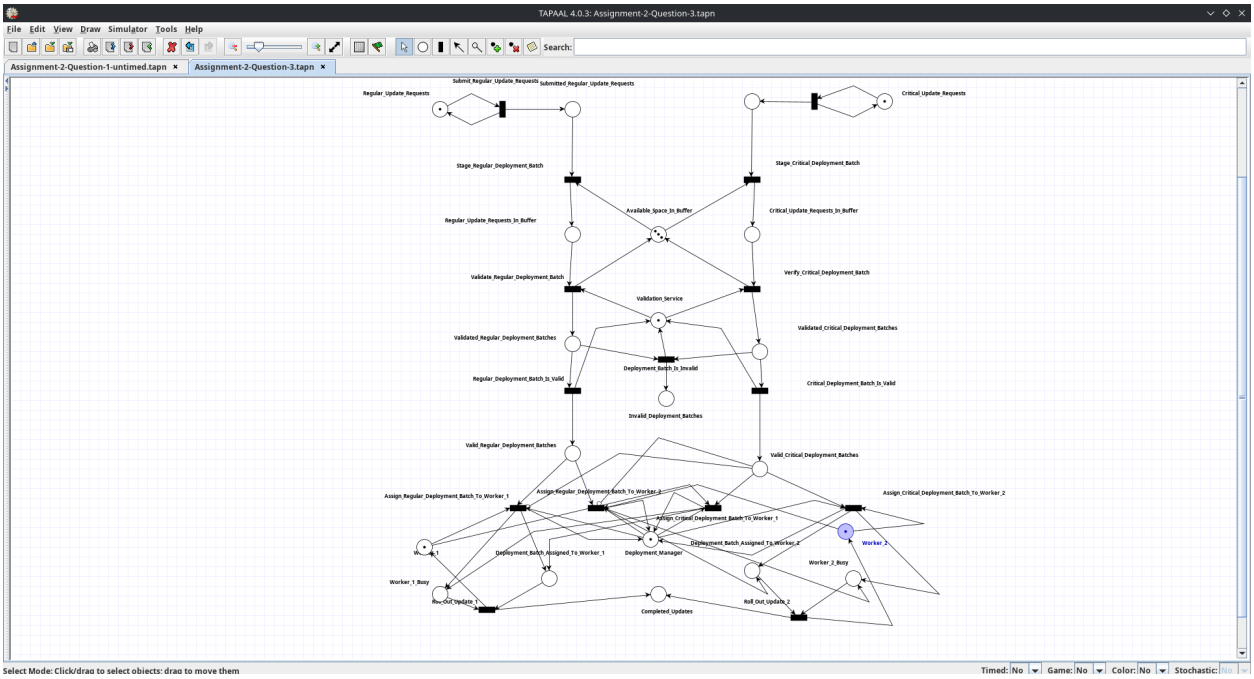

Figure 6: Extended Petri net model with regular/critical updates and inhibitor arcs.

[3 marks]
Question 4

Formulate CTL queries to capture the following requirements. For each requirement:

1. Formulate the CTL expression and explain it in your report;
2. Implement it in TAPAAL;
3. Disclose the result of the query in your report and interpret it.

(a) Priority enforcement – When critical updates are waiting for deployment, regular updates cannot begin deployment.

Solution:

(I) CTL Expression

We need to verify that whenever at least one valid critical deployment batch is waiting, the transitions that assign regular batches to workers are disabled. This is a universal safety property. The CTL expression is:

$$
\begin{aligned}
AG\Big(&\text{TAPN1.Valid_Critical_Deployment_Batches} = 0 \\
&\vee \ \big(\text{TAPN1.Valid_Critical_Deployment_Batches} \geq 1 \\
&\qquad \wedge \ \neg\text{fireable}(\text{TAPN1.Assign_Regular_Deployment_Batch_To_Worker_1}) \\
&\qquad \wedge \ \neg\text{fireable}(\text{TAPN1.Assign_Regular_Deployment_Batch_To_Worker_2})\big)\Big)
\end{aligned}
$$

This reads: "for all reachable states, either no valid critical batches are waiting, or if at least one is waiting, then neither of the regular assignment transitions can fire." The disjunction encodes the implication $P \rightarrow Q$ as $\neg P \vee Q$, where P is "critical batches are waiting" and Q is "regular assignment is disabled."

(II) TAPAAL Result

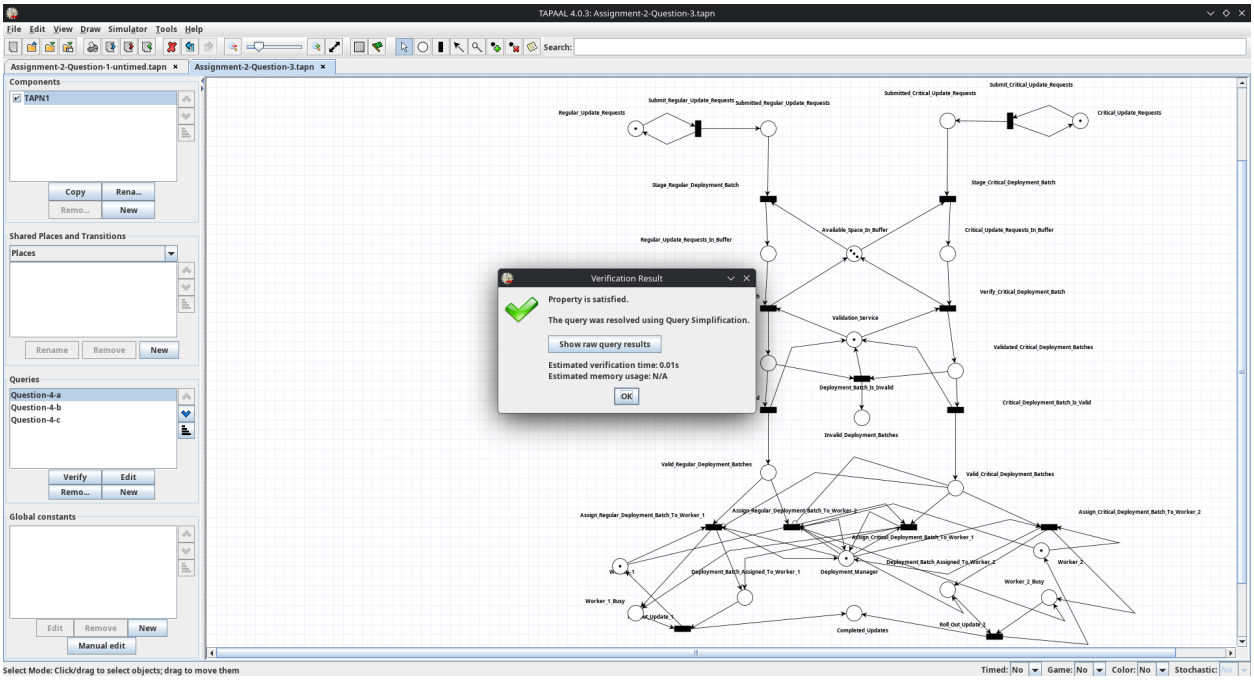

Figure 7: TAPAAL verification result for priority enforcement (Q4a).

(III) Raw Query Output

```
Parameters: --k-bound 13 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta1602806796961113279.xml /tmp/verifyta12823042464755397123.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=13,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: AG ((TAPN1_Valid_Critical_Deployment_Batches == 0) or ((1 <=
    TAPN1_Valid_Critical_Deployment_Batches) and (not (((1 <=
    TAPN1_Valid_Regular_Deployment_Batches) and (0 >= TAPN1_Valid_Critical_Deployment_Batches)
     and (1 <= TAPN1_Worker_1) and (1 <= TAPN1_Deployment_Manager)))) and (not (((1 <=
    TAPN1_Valid_Regular_Deployment_Batches) and (0 >= TAPN1_Valid_Critical_Deployment_Batches)
     and (1 <= TAPN1_Worker_2) and (1 <= TAPN1_Deployment_Manager))))))
RWSTATS PRE:0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,2,0,0,0,0,
RWSTATS POST:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

Query after reduction: true
Query reduction finished after 0.000953 seconds.
```

```
Query size reduced from 31 to 0 nodes ( 100 percent reduction).
FORMULA Question-4-a TRUE TECHNIQUES COLLATERAL_PROCESSING STRUCTURAL_REDUCTION
    QUERY_REDUCTION SAT_SMT LP_APPROX
Query index 0 was solved

Query is satisfied.

Query solved by Query Simplification.

Size of net before structural reductions: 21 places, 15 transitions
Size of net after structural reductions: 0 places, 0 transitions
Structural reduction finished after 3.3e-05 seconds

Net reduction is enabled.
Removed transitions: 15
Removed places: 21
Applications of rule A: 0
Applications of rule B: 0
Applications of rule C: 0
Applications of rule D: 0
Applications of rule E: 0
Applications of rule F: 0
Applications of rule G: 0
Applications of rule H: 0
Applications of rule I: 1
Applications of rule J: 0
Applications of rule K: 0
Applications of rule L: 0
Applications of rule M: 0
Applications of rule N: 0
Applications of rule O: 0
Applications of rule P: 0
Applications of rule Q: 0
Applications of rule R: 0
Applications of rule S: 0
```

(IV) Interpretation

The property is satisfied. This confirms that the inhibitor arcs from Valid_Critical_Deployment_Batches to both regular assignment transitions are functioning as intended. Whenever the Valid_Critical_Deployment place contains at least one token, the inhibitor arcs prevent either regular assignment transition from firing, thereby enforcing the priority policy.

(b) Critical update progress – Critical updates that enter the system eventually complete deployment.

Solution:

(I) CTL Expression

This is a liveness property. Ideally, we would like to verify that whenever a critical batch has been submitted, it is guaranteed to eventually complete deployment. The ideal CTL formulation is:

$$AG\,(\text{TAPN1.Submitted_Critical_Update_Requests} = 0 \;\vee\; AF\,(\text{TAPN1.Completed_Updates} \geq 1))$$

This encodes the implication: "in every reachable state, if a critical batch has been submitted, then on all paths from that state, some batch eventually completes."

However, there are two significant limitations to note:

- This formula contains nested path quantifiers (AG wrapping AF), which is computationally prohibitive for the model checker. TAPAAL was unable to verify this formula in reasonable time.
- The Completed_Updates place is shared between regular and critical batches (since the Petri net is uncolored), so even if the property were satisfied, it would only tell us that some batch completes, not necessarily the critical one. Tracking a specific token through the pipeline is a fundamental limitation of uncolored Petri nets.

As a practical alternative, I instead verify the following non-nested formula:

$$EF\,(\text{TAPN1.Valid_Critical_Deployment_Batches} \geq 1 \;\wedge\; \text{deadlock})$$

This asks: "can the system reach a deadlocked state in which valid critical batches are stuck waiting?" If this property is not satisfied, then no critical batch can ever be permanently trapped in a deadlocked state.

(II) TAPAAL Result

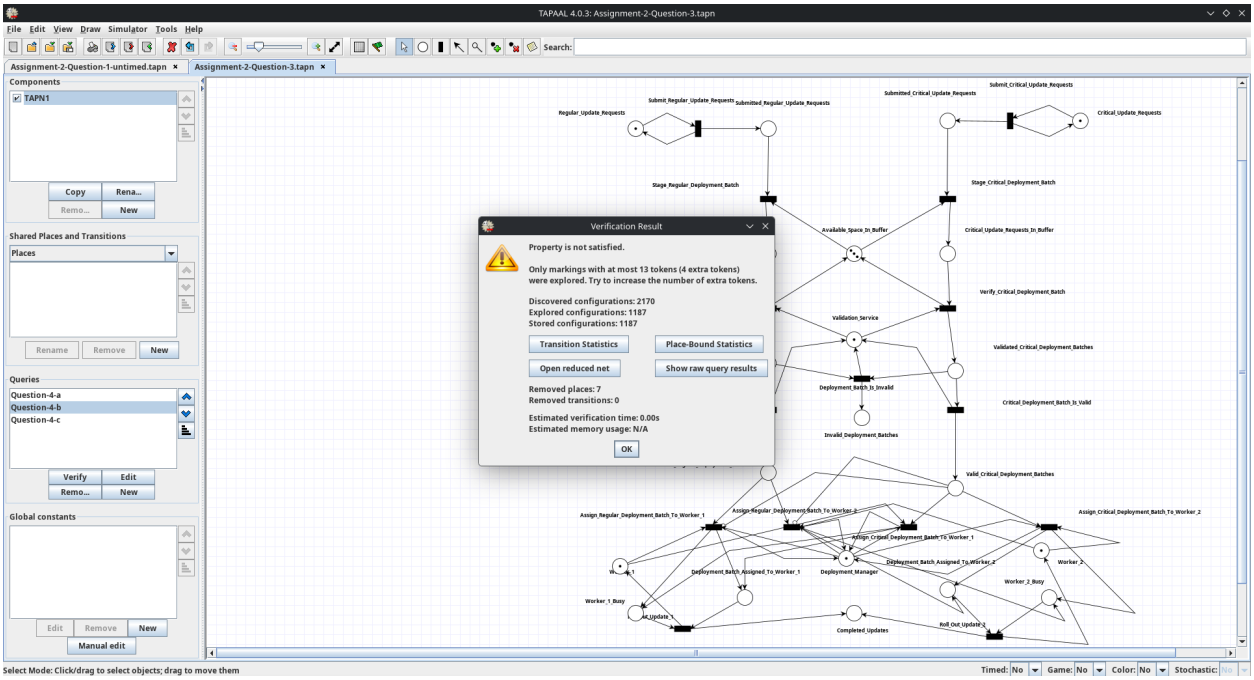

Figure 8: TAPAAL verification result for critical update progress (Q4b).

(III) Raw Query Output

```
Parameters: --k-bound 13 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta15038672009860532788.xml /tmp/verifyta13643882402402069117.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=13,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: EF ((1 <= TAPN1_Valid_Critical_Deployment_Batches) and deadlock)
RWSTATS PRE:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
RWSTATS POST:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

Query after reduction: EF ((1 <= TAPN1_Valid_Critical_Deployment_Batches) and deadlock)
Query reduction finished after 0.00019 seconds.
Query size reduced from 6 to 6 nodes ( 0 percent reduction).
Size of net before structural reductions: 21 places, 15 transitions
Size of net after structural reductions: 14 places, 15 transitions
Structural reduction finished after 2.6e-05 seconds
```

```
Net reduction is enabled.
Removed transitions: 0
Removed places: 7
Applications of rule A: 0
Applications of rule B: 0
Applications of rule C: 2
Applications of rule D: 0
Applications of rule E: 0
Applications of rule F: 2
Applications of rule G: 0
Applications of rule H: 0
Applications of rule I: 0
Applications of rule J: 0
Applications of rule K: 0
Applications of rule L: 0
Applications of rule M: 3
Applications of rule N: 0
Applications of rule O: 0
Applications of rule P: 0
Applications of rule Q: 0
Applications of rule R: 0
Applications of rule S: 0
FORMULA Question-4-b FALSE TECHNIQUES COLLATERAL_PROCESSING STRUCTURAL_REDUCTION
    QUERY_REDUCTION SAT_SMT EXPLICIT STATE_COMPRESSION STUBBORN_SETS
Query index 0 was solved

Query is NOT satisfied.

STATS:
    discovered states: 2170
    explored states:   1187
    expanded states:   1187
    max tokens:        14

TRANSITION STATISTICS
<TAPN1_Submit_Regular_Update_Requests:141><TAPN1_Submit_Critical_Update_Requests:93><
    TAPN1_Stage_Regular_Deployment_Batch:185><TAPN1_Stage_Critical_Deployment_Batch:103><
    TAPN1_Validate_Regular_Deployment_Batch:56><TAPN1_Verify_Critical_Deployment_Batch:108><
    TAPN1_Deployment_Batch_Is_Invalid:0><TAPN1_Regular_Deployment_Batch_Is_Valid:68><
    TAPN1_Critical_Deployment_Batch_Is_Valid:505><
    TAPN1_Assign_Regular_Deployment_Batch_To_Worker_1:222><
    TAPN1_Assign_Regular_Deployment_Batch_To_Worker_2:222><
    TAPN1_Assign_Critical_Deployment_Batch_To_Worker_1:0><
    TAPN1_Assign_Critical_Deployment_Batch_To_Worker_2:0><TAPN1_Roll_Out_Update_1:243><
    TAPN1_Roll_Out_Update_2:223>

PLACE-BOUND STATISTICS
<TAPN1_Submitted_Regular_Update_Requests;1><TAPN1_Submitted_Critical_Update_Requests;1><
    TAPN1_Available_Space_In_Buffer;3><TAPN1_Regular_Update_Requests_In_Buffer;3><
    TAPN1_Critical_Update_Requests_In_Buffer;3><TAPN1_Validation_Service;1><
    TAPN1_Validated_Regular_Deployment_Batches;1><TAPN1_Validated_Critical_Deployment_Batches
    ;1><TAPN1_Valid_Regular_Deployment_Batches;7><TAPN1_Valid_Critical_Deployment_Batches;1><
    TAPN1_Worker_1;1><TAPN1_Worker_2;1><TAPN1_Deployment_Batch_Assigned_To_Worker_1;1><
    TAPN1_Deployment_Batch_Assigned_To_Worker_2;1><TAPN1_Invalid_Deployment_Batches;?><
    TAPN1_Worker_2_Busy;?><TAPN1_Critical_Update_Requests;?><TAPN1_Regular_Update_Requests;?><
    TAPN1_Deployment_Manager;?><TAPN1_Worker_1_Busy;?><TAPN1_Completed_Updates;?>

Spent 0.001826 on verification
```

(IV) Interpretation

The practical formula is not satisfied, meaning the system cannot reach a deadlocked state in which critical batches are stuck waiting for dispatch. Combined with the priority enforcement result from part (a), which guarantees that critical batches are always dispatched

before regular ones, we can argue that critical batches will always eventually make progress.

More specifically, we know from part (a) that regular batches cannot preempt critical ones at the dispatch stage. We also know that the system never reaches a deadlock while critical batches are waiting. And structurally, once a critical batch is dispatched to a worker, the only path forward is through rollout to completion, with no contention that could permanently block it. Taken together, these results provide strong evidence that critical batches will always eventually complete deployment, even though we cannot verify the ideal nested formula directly.

It should be noted, however, that the verification was bounded: TAPAAL explored only markings with at most 13 tokens, so this result holds within that bound but is not a complete proof. The unbounded nature of the net (due to the submission self-loops) makes exhaustive verification impossible with finite token bounds.

(c) Regular update starvation – Determine whether regular updates may be indefinitely delayed if critical updates keep arriving.

Solution:

(I) CTL Expression

This question asks us to determine whether starvation of regular updates is possible. Given the priority policy enforced by the inhibitor arcs, I expect the answer to be yes: if critical batches keep arriving, the inhibitor arcs will continuously block regular batch dispatch. To check this, I verify:

$$EF\,(\text{TAPN1.Valid_Regular_Deployment_Batches} \geq 1 \;\wedge\; \text{TAPN1.Valid_Critical_Deployment_Batches} \geq 1)$$

This asks: "can the system reach a state in which both a valid regular batch and a valid critical batch are waiting for dispatch simultaneously?" If this state is reachable, then by the priority enforcement result from part (a), the regular batch is blocked from being dispatched while the critical batch is present.

(II) TAPAAL Result

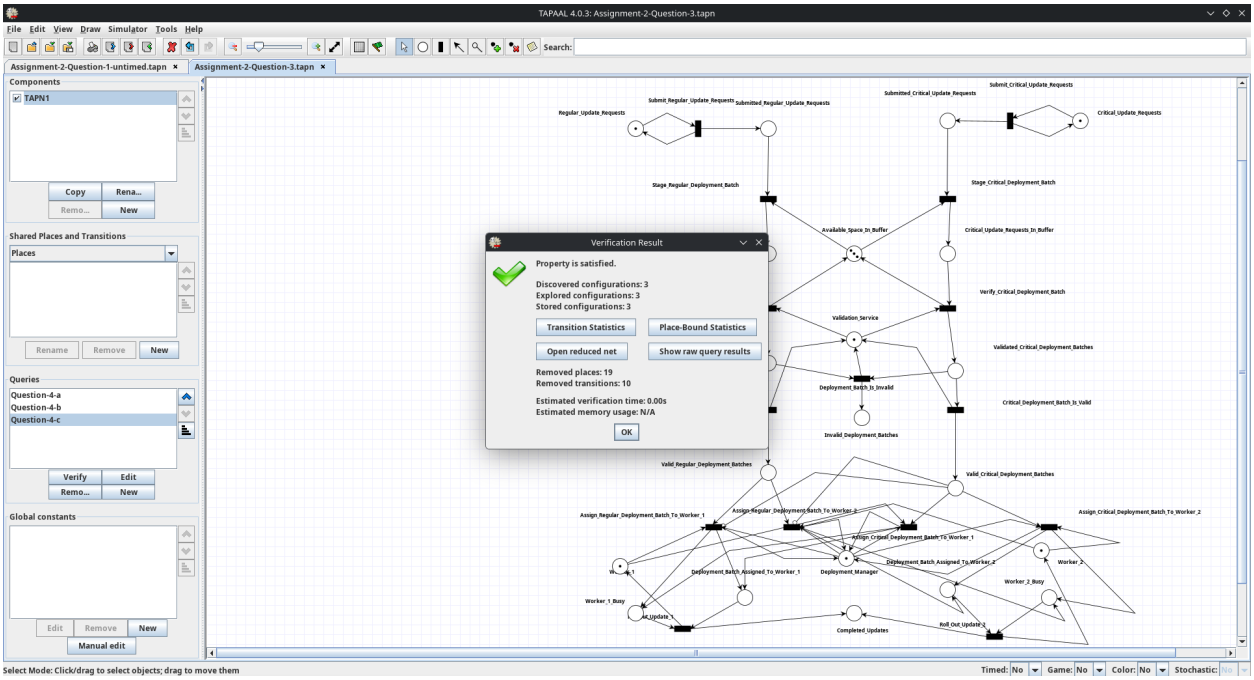

Figure 9: TAPAAL verification result for regular update starvation (Q4c).

(III) Raw Query Output

```
Parameters: --k-bound 13 --search-strategy DFS --reduction 1 --write-reduced /tmp/reduced
    -14913286173355434555.pnml --ctl-algorithm czero --xml-queries 1 /tmp/
    verifyta9183076091680090646.xml /tmp/verifyta12056512263134257071.xml

Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=13,State_Space_Exploration=
    DISABLED,Colored_Structural_Reduction=ALL,Colored_Struct_Red_Timout=30,
    Structural_Reduction=AGGRESSIVE,Struct_Red_Timout=60,Stubborn_Reduction=ENABLED,
    Query_Simplication=ENABLED,QSTimeout=30,Init_Potency=ENABLED,InitPotencyTimeout=10,
    Siphon_Trap=DISABLED,LPSolve_Timeout=10
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX true -> !
    deadlock,!AX p -> EX p,AX false -> deadlock,AX true -> true,EF !deadlock -> !deadlock,EF
    EF p -> EF p,EF AF p -> AF p,EF E p U q -> EF q,EF A p U q -> EF q,EF .. or .. -> EF .. or
     EF ..,AF !deadlock -> !deadlock,AF AF p -> AF p,AF EF p -> EF p,AF .. or EF p -> EF p or
    AF ..,AF A p U q -> AF q,A p U !deadlock -> !deadlock,A deadlock U q -> q,A !deadlock U q
    -> AF q,A p U AF q -> AF q,A p U EF q -> EF q,A p U .. or EF q -> EF q or A p U ..,E p U !
    deadlock -> !deadlock,E deadlock U q -> q,E !deadlock U q -> EF q,E p U EF q -> EF q,E p U
     .. or EF q -> EF q or E p U ..,!! p -> p,F F p -> F p,F p U q -> F q,F p or q -> F p or F
     q,p U F q -> F q,

Query before reduction: EF ((1 <= TAPN1_Valid_Regular_Deployment_Batches) and (1 <=
    TAPN1_Valid_Critical_Deployment_Batches))
RWSTATS PRE:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
RWSTATS POST:0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,

Query after reduction: EF ((1 <= TAPN1_Valid_Regular_Deployment_Batches) and (1 <=
    TAPN1_Valid_Critical_Deployment_Batches))
Query reduction finished after 0.000261 seconds.
Query size reduced from 8 to 8 nodes ( 0 percent reduction).
Size of net before structural reductions: 21 places, 15 transitions
Size of net after structural reductions: 2 places, 5 transitions
```

```
Structural reduction finished after 6.5e-05 seconds

Net reduction is enabled.
Removed transitions: 10
Removed places: 19
Applications of rule A: 2
Applications of rule B: 6
Applications of rule C: 2
Applications of rule D: 1
Applications of rule E: 0
Applications of rule F: 0
Applications of rule G: 0
Applications of rule H: 0
Applications of rule I: 2
Applications of rule J: 0
Applications of rule K: 0
Applications of rule L: 0
Applications of rule M: 7
Applications of rule N: 0
Applications of rule O: 0
Applications of rule P: 0
Applications of rule Q: 0
Applications of rule R: 0
Applications of rule S: 4
FORMULA Question-4-c TRUE TECHNIQUES COLLATERAL_PROCESSING STRUCTURAL_REDUCTION
    QUERY_REDUCTION SAT_SMT EXPLICIT STATE_COMPRESSION STUBBORN_SETS
Query index 0 was solved

Query is satisfied.

STATS:
    discovered states: 3
    explored states:   3
    expanded states:   1
    max tokens:        2

TRANSITION STATISTICS
<TAPN1_Submit_Regular_Update_Requests:1><TAPN1_Deployment_Batch_Is_Invalid:1><
    TAPN1_Assign_Regular_Deployment_Batch_To_Worker_1:0><
    TAPN1_Assign_Regular_Deployment_Batch_To_Worker_2:0><
    TAPN1_Assign_Critical_Deployment_Batch_To_Worker_1:0><TAPN1_Roll_Out_Update_1:?><
    TAPN1_Critical_Deployment_Batch_Is_Valid:?><TAPN1_Roll_Out_Update_2:?><
    TAPN1_Stage_Regular_Deployment_Batch:?><TAPN1_Regular_Deployment_Batch_Is_Valid:?><
    TAPN1_Verify_Critical_Deployment_Batch:?><TAPN1_Validate_Regular_Deployment_Batch:?><
    TAPN1_Stage_Critical_Deployment_Batch:?><TAPN1_Submit_Critical_Update_Requests:?><
    TAPN1_Assign_Critical_Deployment_Batch_To_Worker_2:?>

PLACE-BOUND STATISTICS
<TAPN1_Valid_Regular_Deployment_Batches;1><TAPN1_Valid_Critical_Deployment_Batches;1><
    TAPN1_Validated_Critical_Deployment_Batches;?><TAPN1_Invalid_Deployment_Batches;?><
    TAPN1_Worker_2_Busy;?><TAPN1_Validated_Regular_Deployment_Batches;?><
    TAPN1_Critical_Update_Requests_In_Buffer;?><TAPN1_Deployment_Batch_Assigned_To_Worker_1
    ;?><TAPN1_Regular_Update_Requests_In_Buffer;?><TAPN1_Submitted_Critical_Update_Requests
    ;?><TAPN1_Critical_Update_Requests;?><TAPN1_Submitted_Regular_Update_Requests;?><
    TAPN1_Validation_Service;?><TAPN1_Available_Space_In_Buffer;?><TAPN1_Worker_2;?><
    TAPN1_Regular_Update_Requests;?><TAPN1_Worker_1;?><TAPN1_Deployment_Manager;?><
    TAPN1_Worker_1_Busy;?><TAPN1_Completed_Updates;?><
    TAPN1_Deployment_Batch_Assigned_To_Worker_2;?>

Spent 0.000494 on verification
```

(IV) Interpretation

The property is satisfied, confirming that the system can indeed reach a state where regular

batches are waiting alongside critical batches. From part (a), we know that in such a state, the regular batch assignment transitions are disabled. Since the Critical_Update_Requests place has a self-loop that allows unbounded submission of new critical requests, the system can continuously produce new valid critical batches. Each time a critical batch is dispatched and the Valid_Critical_Deployment_Batches place is momentarily emptied, a new critical batch can arrive and refill it before a regular batch has a chance to be dispatched.

Therefore, regular update starvation is possible in this model. This is a direct consequence of the strict priority policy: the inhibitor arcs provide no fairness guarantee. In a real system, this could be mitigated by introducing mechanisms such as aging (promoting regular batches to critical after a timeout) or reserving one worker exclusively for regular updates, but such mechanisms are beyond the scope of this model.

[2 marks]
Question 5

Reflect on this assignment and the overall module (MBT and Petri nets) briefly, e.g., by elaborating on the following questions, or your own points.

- How do you see verification being useful in your future industry job? What did you learn about the complexity of proving correctness (verification) vs increasing our confidence in the correctness (validation) of the program? Which one would you use in your daily professional activities and why?
- What is the role of a tool in mathematically heavy methods, such as verification with Petri nets? Does the existence of a good tool make a difference in the adoption of the formal verification techniques, or is it mostly irrelevant?
- What is the link between verification and validation.
- What was the hardest part of this assignment and why?

Solution:

The hardest part of this assignment was, without question, formulating and verifying the CTL queries for Questions 2 and 4. While the Petri net modelling itself was fairly mechanical once I understood the system requirements, the CTL formulation process exposed several challenges that I did not anticipate going in.

The first major difficulty was the ambiguity of the natural language descriptions in the assignment. Several of the requirements could be reasonably interpreted in more than one way. For instance, "critical updates that enter the system eventually complete deployment" could mean that completion is guaranteed on all paths (AF), or that completion is always at least possible (EF). The choice between these two readings changes not only the formula but also what the result actually tells you about the system. I found myself spending more time reasoning about what the question was really asking than I did constructing the formula itself.

The second challenge was computational. Several of the CTL formulas I came up with were correct in principle but unverifiable in practice. Specifically, any formula with nested path quantifiers (such as AG wrapping AF, or AG wrapping EF) caused TAPAAL's verifier to either run indefinitely or nearly crash my machine. This was especially frustrating for Question 4b, where the ideal formula for critical update progress required exactly this kind of nesting. I had to resort to weaker, non-nested alternatives and supplement them with structural reasoning across multiple query results to build a convincing argument. This taught me an important lesson: the gap between what you can express in a logic and what you can actually verify with a tool is significant, and working within a tool's limitations is itself a skill.

The third difficulty was the limitation of working with uncoloured Petri nets. Since all tokens in an uncoloured net are anonymous, there is no way to track a specific deployment

batch from submission through to completion. This meant that even when I could verify that some batch eventually completes, I could not verify that a particular critical batch does. The Completed_Updates place accumulates tokens from both regular and critical pipelines indistinguishably. A coloured Petri net would have allowed me to tag tokens by type and trace them individually, but that was outside the scope of this assignment.

On the broader questions posed by this module: I think the most important takeaway is the distinction between verification and validation. Verification asks whether a model satisfies a specification with mathematical certainty, while validation asks whether the model itself faithfully represents the real system. In this assignment, I could verify properties of my Petri net, but whether my net correctly captures the deployment pipeline described in the problem is a separate question entirely. Both are necessary, and neither is sufficient on its own.

In a professional setting, I expect I would rely primarily on validation (testing, code review, integration checks) for day-to-day work, simply because formal verification is expensive and the tooling is often impractical for large systems. That said, for safety-critical components where correctness is non-negotiable, formal methods would be well worth the investment. This assignment demonstrated that clearly: the properties I could verify gave me genuine confidence in my model's correctness, far beyond what manual inspection alone would provide.

Finally, on the role of tooling: TAPAAL made this entire exercise feasible. Without it, verifying even simple CTL properties on a net of this size would require exhaustive manual state space exploration, which is obviously impractical. At the same time, the tool's limitations (bounded verification, inability to handle nested quantifiers efficiently) were themselves a source of frustration. Good tooling does not just make formal methods accessible, it largely determines whether they get adopted at all.